

ДОКУМЕНТАЦИЈА

**Тема: Демо апликација за детекција и
броење на автомобили**

Предмет: Дигитално процесирање на слика

Изработила: Софија Андоновска 231012

Ментор: Ивица Димитровски

Датум: 13.2.2026

Содржина

1. Цел на проектот и краток преглед.....	4
1.1 Вовед.....	4
1.2 Цел.....	4
1.3 Како работи апликацијата	5
1.4 Кои технологии се користат.....	5
1.5 Архитектура на апликацијата	6
2.Објаснување на YOLO	7
2.1 Што е YOLO?.....	7
2.2 Како функционира YOLO?.....	7
2.3 Grid division	7
2.4 Предвидување за секоја ќелија	8
2.5 Non-maximum suppression.....	8
2.6 Confidence	9
2.7 Class score.....	9
2.8 Threshold	9
2.9 Intersection over Union (IoU).....	10
2.10 Bounding box формат	11
3. Опис на counting logic.....	11
3.1 Line crossing logic	11
3.2 Практични предизвици при позиционирање на линијата	11
3.3 Подобрување на точноста на броењето со користење на centroid.....	12
4. Опис на имплементираните алгоритми.....	13
4.1 Алгоритам за следење на возила (Vehicle Tracking Algorithm)	13
4.2 Примарен метод за детекција на преминување на линијата	14
4.3 Прв резервен метод за детекција на преминување	14
4.4 Втор резервен метод за детекција на преминување.....	14
4.5 Алгоритам за чистење на стари tracks.....	15
4.6 Алгоритам за визуелизација	15
4.7 Главниот обработувачки циклус	16

4.8 Алгоритмот за категоризација на возила по тип	16
4.9 Спечување на дупликатно броење (Double Counting Prevention).....	17
5. Дијаграми	18
5.1 Data flow diagram.....	18
5.2 Activity diagram	19
6.Референци	20

1. Цел на проектот и краток преглед

1.1. Вовед

Оваа демо апликација е имплементирана со цел автоматизирано следење и броење на возила во видео записи, користејќи современи техники од областа на компјутерската визија и машинското учење. Главната цел на апликацијата е да овозможи прецизна анализа на сообраќајниот тек преку автоматско детектирање, следење и броење на автомобили и други типови возила, без потреба од човечка интервенција.

Апликацијата комбинира **YOLOv8** (You Only Look Once, верзија 8) модел за детекција на објекти со вградениот tracking механизам на YOLOv8 и кастомна логика за броење на возила низ повеќе последователни видео рамки. YOLOv8 моделот се користи за детекција и класификација на возилата во секоја рамка од видеото, при што за секое детектирано возило се добиваат информации како што се координати на bounding box, класа на објектот (car, truck, bus, итн.) и confidence вредност која ја претставува сигурноста на детекцијата.

Со цел да се овозможи следење на возилата низ времето, апликацијата не се потпира само на детекција во поединечни рамки, туку имплементира алгоритам за следење кој користи **центроиди (centroids)** на bounding box-овите. Центроидите се користат за поврзување на истите возила во различни рамки и за доделување на уникатни идентификатори (ID) на секое возило. Овој пристап овозможува стабилно следење и спречува повеќекратно броење на исто возило.

Броењето на возилата се извршува преку логика за преминување на **виртуелна линија (line-crossing logic)**, каде што секое возило се брои само еднаш во моментот кога неговиот центроид ја преминува дефинираната линија во одредена насока. Со ова се обезбедува прецизно броење дури и во услови на густ сообраќај, преклопување на возила или краткотрајни загуби на детекција.

1.2. Цел

Главната цел на апликацијата е да детектира возила (автомобили, камиони, автобуси, мотоцикли) во видео записи и точно да ги брои колку возила поминале низ одредена виртуелна линија во видеото. Ова е многу корисна функционалност за:

- Анализа на сообраќајот
- Статистика за сообраќајниот проток
- Автоматизација на броење на возила

1.3. Како работи апликацијата

Апликацијата работи на следниов начин:

1. **Вчитување на Видео:** Апликацијата вчитува видео фајл.
2. **Обработка на Фрејмови:** Секој фрејм од видеото се обработува поединечно.
3. **Детекција со YOLO:** YOLOv8 моделот го анализира секој фрејм и ги детектира сите возила присутни во фрејмот.
4. **Следење (Tracking):** На секое детектирано возило му се доделува уникатен ID кој се одржува низ фрејмовите, овозможувајќи следење на движењето на возилото.
5. **Пресметување на Центроиди:** За секое детектирано возило се пресметува центроид (центар на bounding box-от), кој се користи за точно одредување на позицијата на возилото.
6. **Детекција на Преминување:** Апликацијата проверува дали центроидот на возилото ја преминал виртуелната линија за броење.
7. **Броење:** Кога возилото ќе ја премине линијата, се зголемува соодветниот бројач.
8. **Визуелизација:** Резултатите се прикажуваат во реално време со исцртување на bounding boxes, центроиди, линијата за броење и бројачите.

1.4. Кои технологии се користат

YOLOv8 (You Only Look Once верзија 8)

- State-of-the-art модел за детекција на објекти
- Обезбедува брза и прецизна детекција во реално време
- Вграден tracking механизам за следење на објекти

OpenCV (Open Source Computer Vision Library)

- Библиотека за обработка на слики и видео
- Користена за читање/пишување на видео, исцртување на визуелни елементи
- Верзија: $\geq 4.8.0$

NumPy

- Библиотека за нумерички пресметки
- Користена за манипулација на координати и податоци
- Верзија: $\geq 1.24.0$

Ultralytics

- Библиотека која го обезбедува YOLOv8 моделот
- Верзија: $\geq 8.0.0$

Python 3.8

- Програмски јазик за имплементација

1.5. Архитектура на апликацијата

Апликацијата е структурирана како објектно-ориентирана класа `VehicleCounter`

VehicleCounter

- `init()` - Иницијализација
- `update_tracking()` - Следење и броење
- `draw_detections()` - Визуелизација на детекции
- `draw_counters()` - Прикажување на бројачи
- `draw_counting_line()` - Исцртување на линија
- `process_video()` - Главна обработка

Помошни функции

- `has_crossed_line_direction()`
- `has_crossed_in_history()`
- `cleanup_old_tracks()`
- `increment_vehicle_type_counter()`

2. Објаснување на YOLO (You only look once)

2.1.Што е YOLO?

YOLO (You Only Look Once) е револуционерен алгоритм за детекција на објекти кој го трансформираше полето на компјутерската визија. За разлика од традиционалните методи кои користат "sliding window" пристап (каде секој дел од сликата се проверува одделно), YOLO ја дели сликата на решетка и во еден помин го детектира сите објекти.

Клучната разлика помеѓу традиционалните методи за детекција на објекти и YOLO (You Only Look Once) алгоритмот се состои во начинот на обработка на сликата. Традиционалните пристапи најчесто користат повеќекратни поминувања низ сликата, при што се анализираат различни региони со посебни класификатори, што резултира со значително поголемо време на извршување и помала ефикасност, особено кај видео записи во реално време. Наспроти тоа, YOLO алгоритмот ја обработува целата слика во еден единствен фрејм, користејќи длабока невронска мрежа која истовремено предвидува bounding box-ови, класи на објекти и нивната сигурност. Овој пристап овозможува значително побрза детекција со минимално доцнење, што го прави YOLO особено погоден за апликации кои бараат работа во реално време, како што е токму овој системи за следење и броење на возила.

2.2.Како функционира YOLO?

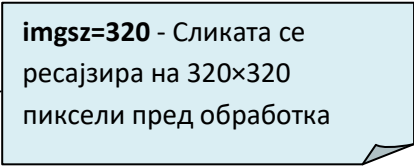
YOLO работи на принципот на "регресија на целиот проблем". Наместо да користи сложени pipeline-и, YOLO директно предвидува bounding boxes и веројатности за класи во еден помин.

2.3.Grid Division

Сликата се дели на $S \times S$ решетка. Во YOLOv8, ова обично е 13×13 , 26×26 , или 52×52 зависно од резолуцијата на влезната слика. Секоја ќелија од решетката е одговорна за детекција на објекти чиј центар паѓа во таа ќелија.

Во 'vehicle_detection.py', поделбата на решетка се случува внатрешно од YOLOv8 моделот:

```
results = self.model.track(  
    frame,  
    persist=True,  
    verbose=False,  
    imgsz=320,  
    conf=self.confidence_threshold,
```



imgsz=320 - Сликата се ресајзира на 320x320 пиксели пред обработка

```
device='cpu',  
max_det=20,  
agnostic_nms=True,  
stream=False  
)
```

2.4.Предвидување за секоја ќелија

За секоја ќелија од мрежата на сликата, YOLO алгоритмот предвидува повеќе информации поврзани со потенцијалните објекти што се наоѓаат во таа област. Најпрво, се предвидуваат координатите на bounding box-от, кои ги вклучуваат вредностите x и y што го претставуваат центарот на bounding box-от релативно во рамките на соодветната ќелија, како и ширината и висината на bounding box-от кои се изразени релативно на целокупната големина на сликата. Овие параметри овозможуваат прецизно лоцирање и големинско опишување на детектираниот објект.

Покрај геометриските информации, YOLO предвидува и confidence score, кој ја претставува сигурноста на моделот дека во дадената ќелија навистина постои објект. Овој резултат се пресметува како производ од веројатноста дека објектот постои ($P(\text{object})$) и Intersection over Union (IoU) вредноста помеѓу предвидениот и вистинскиот bounding box. Confidence score вредноста се движи во опсег од 0.0 до 1.0, при што повисоките вредности означуваат поголема сигурност и подобро совпаѓање со вистинската позиција на објектот.

Дополнително, за секој детектиран објект се пресметуваат и веројатностите за припадност кон одредени класи (class probabilities), како што се автомобил, камион, автобус и мотор. Секоја класа има своја веројатност во опсег од 0.0 до 1.0, што му овозможува на системот не само да детектира објекти, туку и прецизно да ги класифицира според нивниот тип, што е од особена важност за понатамошна анализа и селективно броење на возила.

Имплементацијата во нашиот код се потпира на објектот results.bboxes, преку кој се пристапува до предвидените координати на детектираните возила. Конкретно, во функциите **update_tracking()** и **draw_detections()**, ги извлекуваме **Bounding Box** координатите во формат (x_1, y_1, x_2, y_2). Овие вредности потоа се конвертираат во цели броеви за да се пресметаат координатите на центарот на секој детектиран објект. Центарот, односно неговиот **centroid**, се добива со пресметување на средната вредност помеѓу крајните точки. Оваа точка е клучна за понатамошно следење на движењето на возилата низ кадрите.

2.5.Non-Maximum Suppression (NMS)

Бидејќи повеќе ќелии може да детектираат ист објект, YOLO користи NMS за отстранување на дупликати.

NMS (Non-Maximum Suppression) не е рачно имплементиран во овој код, туку го обработува YOLO моделот од ultralytics внатрешно. Во методот process_video (линија 431-

441), при повикот на `model.track()`, се користи параметарот `agnostic_nms=True` (линија 439), што активира `class-agnostic NMS`. Ова значи дека YOLO автоматски ги сортира детекциите по `confidence score`, пресметува `IoU` (`Intersection over Union`) помеѓу преклопувачките `bounding boxes`, и ако `IoU` надминува праг (обично 0.5), ги отстранува детекциите со понизок `confidence`, оставајќи ги само најдобрите детекции. `Class-agnostic NMS` значи дека преклопувачките боксови се супресираат без разлика на класата, што е корисно кога различни класи може да се преклопуваат. Дополнително, параметарот `conf=self.confidence_threshold` (линија 436) филтрира детекции под минималниот `confidence` праг пред `NMS`, а `max_det=20` (линија 438) ограничува максимален број детекции по фрејм, што ја оптимизира перформансата на `NMS` процесот.

2.6. Confidence

`Confidence` (Доверба) е веројатноста дека детектираниот објект е навистина тоа што моделот го идентификува. Во нашиот код, `confidence` се зема од YOLO резултатите на линија 119: `conf = results.bboxes.conf[i].item()`. Вредноста е помеѓу 0.0 и 1.0; повисоко значи поголема сигурност. На линија 126 се филтрираат детекциите со ниска доверба: `if conf < self.confidence_threshold: continue`, што значи дека детекциите под прагот се игнорираат. Истото се применува и во `draw_detections` на линија 272, каде се прикажуваат само детекциите што го поминуваат прагот. На линија 310, `confidence` се прикажува во `label`-от: `label = f"ID:{track_id} {vehicle_name}: {conf:.2f}"`, што овозможува визуелна проверка на сигурноста на секоја детекција.

2.7. Class score

`Class Score` (Резултат на класа) е веројатноста дека објектот припаѓа на одредена класа. Во нашиот код, класата се идентификува преку `class ID` на линија 118: `cls_id = int(results.bboxes.cls[i].item())`. Класите се дефинирани на линија 25: `self.vehicle_classes = [2, 3, 5, 7]` (`car`, `motorcycle`, `bus`, `truck`). На линија 122 се проверува дали детекцијата е од една од овие класи: `if cls_id not in self.vehicle_classes: continue`. Името на класата се мапира на линија 304-307, каде се користи `self.class_names[cls_id]` за приказ на името. `Class Score` е вградено во YOLO; овде се користи `class ID` за филтрирање и категоризација на возилата.

Со користење на `class score` и `class ID`, системот не се ограничува само на детекција на присуство на возила, туку овозможува и класификација според типот на возилото (автомобил, мотоцикл, автобус или камион). Ова овозможува попрецизна анализа на сообраќајот, како што се броење по категории, статистика по тип на возила и потенцијално проширување на системот со посебни бројачи за секоја класа.

2.8. Threshold

`Threshold` (Праг) е минималната вредност на `confidence` под која детекциите се отфрлаат. Во нашиот код, `threshold` се иницијализира на линија 12 како параметар: `confidence_threshold=0.5`, и се чува како атрибут на линија 23: `self.confidence_threshold = confidence_threshold`. Се користи на линија 126 за

филтрирање: `if conf < self.confidence_threshold: continue`, и на линија 272 во `draw_detections`. Исто така, се пренесува директно на YOLO моделот на линија 436: `conf=self.confidence_threshold`, што ги филтрира детекциите на ниво на моделот. Прагот може да се постави преку командна линија на линија 494: `--confidence`, со стандардна вредност 0.5. Повисок праг (на пр. 0.7) дава помалку, но посигурни детекции; понизок праг (на пр. 0.3) дава повеќе детекции, но со поголема веројатност за грешки.

Confidence threshold се користи со цел да се подобри сигурноста на детектираните објекти. Овој праг служи за филтрирање на лажни позитиви, бидејќи YOLO понекогаш може да детектира објекти кои реално не постојат или се резултат на шум во сликата. Детекциите со низок confidence не се доволно сигурни и затоа се отстрануваат. Со елиминирање на слабите детекции се постигнува подобрување на прецизноста, бидејќи системот се фокусира само на релевантни и сигурни објекти. Дополнително, намалувањето на бројот на детекции директно води до помала пресметковна сложеност, односно помалку обработка во следните чекори како tracking, пресметка на centroid и line crossing логика.

Threshold	Број на детекции	Лажни позитиви	Прецизност	Recall
0.3	Многу	Високо	Ниско	Високо
0.5	Средно	Средно	Средно	Средно
0.7	Мало	Ниско	Високо	Ниско

2.9. Intersection over Union (IoU)

Intersection over Union (IoU) е метрика што мери преклопување помеѓу два bounding box-a. Се пресметува како однос помеѓу површината на пресекот (intersection) и површината на унијата (union) на двата бокса. На пример, ако имаме два bounding box-a:

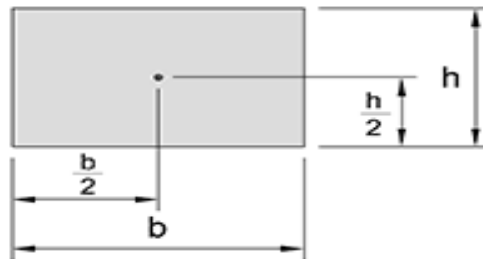
- Бокс А со координати (100, 100, 200, 200)
- Бокс В со координати (150, 150, 250, 250),

пресекот е правоаголникот (150, 150, 200, 200) со површина $50 \times 50 = 2500$ пиксели, додека унијата е правоаголникот (100, 100, 250, 250) со површина $150 \times 150 = 22500$ пиксели.

Со тоа, $\text{IoU} = 2500 / 22500 = 0.11$, што укажува на мало преклопување. Во YOLO, IoU се користи во Non-Maximum Suppression за отстранување на дупликати: ако два bounding box-a имаат IoU поголем од 0.5, се сметаат за дупликати и се зачувува само оној со повисок confidence score. Во нашиот код, ова се постигнува со параметарот `agnostic_nms=True` во методот `process_video` (линија 439), што значи дека NMS се применува независно од класата на објектот, односно дупликати од иста класа се отстрануваат за да се избегне премногу детекции за истиот објект.

2.10. Bounding Box формат

YOLO враќа координати на bounding boxes во хуху формат, што значи $(x1, y1, x2, y2)$, каде $x1$ и $y1$ се координатите на горниот лев агол, а $x2$ и $y2$ се координатите на долниот десен агол на правоаголникот. Во нашиот код, на линија 136 се извлекуваат координатите преку `box = results.bboxes.xxyy[i].cpu().numpy()`, по што се конвертираат во цели броеви со `x1, y1, x2, y2 = box.astype(int)`. Овие координати се користат за пресметување на centroid (центар) на објектот на линија 138-139, каде се пресметуваат `center_x = (x1 + x2) // 2` и `center_y = (y1 + y2) // 2`. Centroid-от е клучен за детекција на преминување на линијата за броење, бидејќи се следи неговата позиција низ фрејмовите за да се одреди дали возилото ја поминало линијата.



3. Опис на counting logic

3.1. Line crossing logic

Line crossing претставува процес на детекција при кој се утврдува дали одреден објект (на пр. возило) ја има преминато виртуелната линија за броење. Во имплементацијата, оваа линија е дефинирана како хоризонтална линија на одредена **Y координата**, која се пресметува како процент од висината на фрејмот. Линијата се исцртува со користење на OpenCV функцијата `cv2.line`, при што служи како референтна граница за броење. За правилно да се детектира преминувањето, неопходно е да се следи движењето на објектот низ последователни фрејмови, па затоа се користат `prev_centroid` и `curr_centroid`. **prev_centroid** ја претставува позицијата на објектот во претходниот фрејм, додека **curr_centroid** ја означува неговата моментална позиција. Со споредување на овие две позиции во однос на Y координатата на линијата, системот може сигурно да утврди дали објектот ја преминал линијата и со тоа да регистрира валиден настан за броење.

3.2. Практични предизвици при позиционирање на линијата

Во текот на развојот беше забележан практичен проблем поврзан со позиционирањето на виртуелната линија за броење. Првично, линијата беше поставена повисоко во фрејмот, што доведуваше до пропуштање на некои возила во ситуации кога повеќе возила се движат едно зад друго на мало меѓусебно растојание. Во такви случаи, возилото кое се

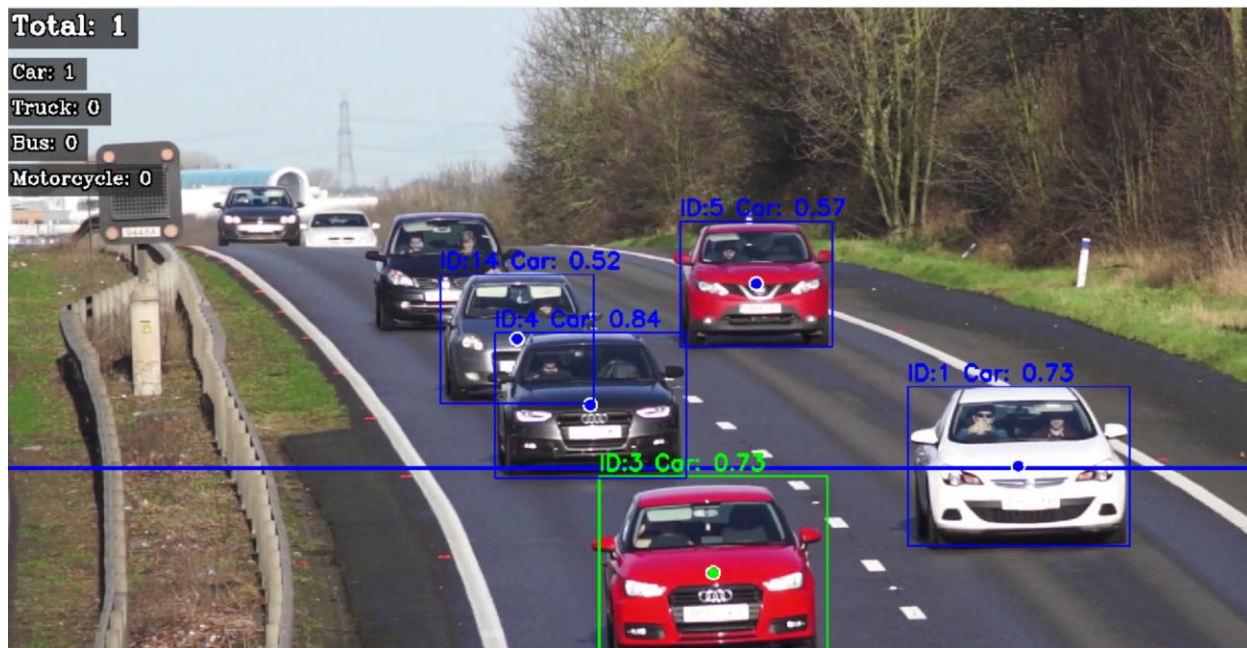
наоѓаше позади не секогаш беше навремено детектирано или следено, поради преклопување и недоволна стабилност на centroid-ите во раната фаза на движењето.

За да се надмине овој проблем, линијата за броење беше поместена малку под средината на фрејмот. Како што возилата се приближуваат кон камерата, нивното меѓусебно растојание во сликата се зголемува, што овозможува појасна и постабилна детекција и следење на секое возило поединечно. Со ова позиционирање, системот покажува поголема сигурност при детекција на последователни возила и значително се намалува бројот на пропуштени настани за броење.

3.3.Подобрување на точноста на броењето со користење на centroid

Дополнителен предизвик при имплементацијата беше обезбедување на точно броење на сите детектирани возила. Во почетната верзија на системот, околу секое возило се исцртуваше bounding box (правоаголник), но броењето не секогаш беше прецизно, особено во случаи кога повеќе возила беа блиску едно до друго или кога доаѓаше до преклопување на детекциите. Во такви ситуации, системот можеше да регистрира повеќе или помалку настани од реалниот број на возила.

За да се подобри точноста, беше воведен пристап базиран на **centroid**, односно точка која ја претставува средината на детектираното возило. Наместо броење врз основа на целиот bounding box, броењето се извршува кога centroid-от на возилото ќе ја премине виртуелната линија за броење. Со овој пристап, секое возило се брои точно еднаш, во моментот кога неговата централна точка ја преминува линијата, што резултира со значително постабилен и попрецизен бројач.



Слика бр.1

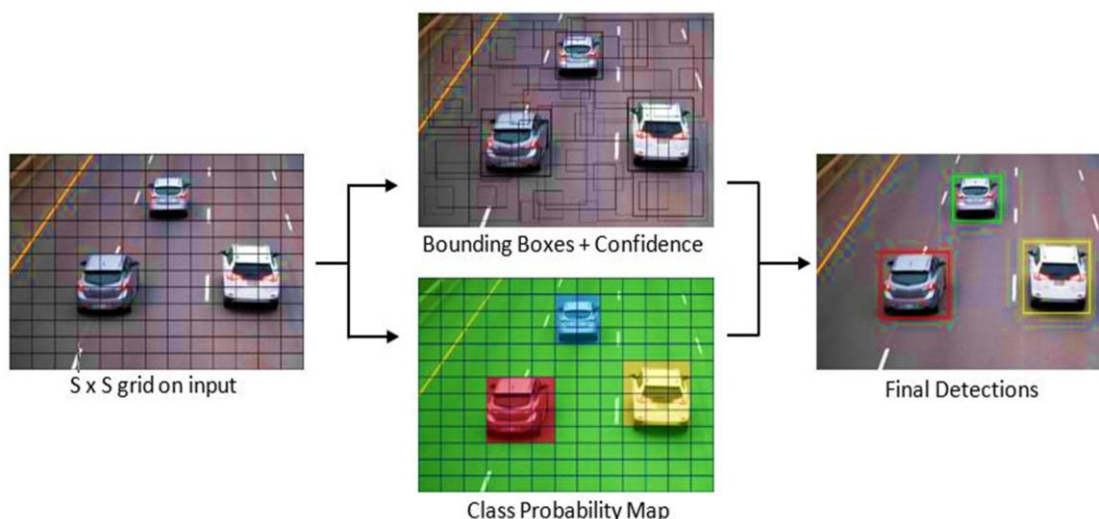
На сликата (Слика бр.1) се прикажани возила кои се движат низ кадарот, со **центроиди означени како точки**. Виртуелната линија за броење е поставена малку подолу од центарот на фрејмот, што овозможува точна детекција на преминување дури и кога повеќе возила се блиску едно до друго. Centroid-ите ја претставуваат средината на секое возило и се користат за правилно броење кога ќе ја преминат линијата.

4. Опис на имплементираниите алгоритми

4.1.Алгоритам за следење на возила (Vehicle Tracking Algorithm)

Алгоритмот за следење користи YOLOv8 вградени track IDs и локална логика за одржување на историја на позиции. Во методот **update_tracking()** (линија 94-217), за секој фрејм се обработуваат детекциите од YOLO. За секое детектирано возило се извлекуваат: class ID (линија 118), confidence score (линија 119), tracking ID (линија 131) и координати на bounding box (линија 136-137). Од координатите се пресметува centroid како средна точка на правоаголникот (линија 138-140). Ако track ID не постои во tracked_vehicles, се креира нов запис со почетни вредности: prev_centroid, curr_centroid, листа positions за историја, class, crossed, frames_seen и first_seen_y (линија 147-155). За постоечки возила, се ажурираат prev_centroid и curr_centroid, се додава новата позиција во историјата (линија 161-165), и се ограничува историјата на max_history (линија 166-167). Ова овозможува следење на движење низ фрејмовите и детекција на преминување на линијата.

Битно е да водиме евиденција за track IDs бидејќи тие овозможуваат секое возило да се идентификува единствено низ последователни фрејмови, што е критично за точно следење и броење, особено кога повеќе возила се движат блиску едно до друго. Track IDs овозможуваат да се одржи историја на позиции и да се одреди дали centroid-от на возилото ја преминал линијата за броење, без ризик од дуплирање или пропуштање настани.



4.2.Примарен метод за детекција на преминување на линијата (Direction-Based Line Crossing Detection)

Примарниот метод за детекција на преминување е **has_crossed_line_direction()** (линија 57-70). Алгоритмот споредува Y координати на `prev_centroid` и `curr_centroid` со `line_y`. Преминување од горе надолу се детектира кога `prev_y < line_y` и `curr_y >= line_y` (линија 65), а од доле нагоре кога `prev_y > line_y` и `curr_y <= line_y` (линија 68).

Ако се детектира преминување:

- возилото се означува како `crossed`,
- се додава во `counted_ids` за спречување на повторно броење,
- се зголемуваат соодветните бројачи,
- се повикува `_increment_vehicle_type_counter()` за категоризација (линија 179-185).

Овој метод е ефикасен кога детекцијата е стабилна и движењето е континуирано. Клучното значење е што користи `centroid`-от на секое возило и `track ID` за да осигура дека секое возило се брои точно еднаш, дури и во ситуации со повеќе возила едно зад друго. Со ова се минимизираат грешки при броење и се обезбедува прецизна статистика по тип на возило.

4.3.Прв резервен метод за детекција на преминување (First Backup Method: Initial Position Comparison)

Првиот резервен метод (линија 187-203) се активира кога примарниот метод нема да успее, особено во случаи кога возилото е детектирано по преминувањето на линијата. Алгоритмот споредува `first_seen_y` (првата Y позиција на `centroid`-от) со моменталната `center_y`.

- Ако возилото започнало над линијата (`first_y < line_y`) и сега е подолу (`center_y > line_y + 20`), се смета дека ја преминало линијата (линија 192).
- Истото важи и за обратна насока (линија 198).

Прагот од 20 пиксели ја намалува веројатноста за лажни позитиви. Овој метод се користи само ако возилото не е веќе избројано и има барем 2 фрејмови видено (`frames_seen >= 2`).

Овој резервен метод обезбедува сигурност во броењето во ситуации каде `centroid`-от на возилото е детектиран по преминувањето на линијата или кога примарниот метод не може веднаш да детектира `crossing`. Со тоа се зачувува точноста на системот дури и при неправилни или краткотрајни детекции.

4.4.Втор резервен метод за детекција на преминување (Second Backup Method: Position History Analysis)

Вториот резервен метод **has_crossed_in_history()** (линија 72-92) анализира историја на позиции на `centroid`-от. Алгоритмот извлекува Y координати од `positions` (линија 81), ги

наоѓа $\min_y$ и $\max_y$ (линија 85-86), и проверува дали опсегот ја покрива линијата: $\min_y < \text{line_y}$ и $\max_y > \text{line_y}$ (линија 89). Ако е исполнето, се смета дека возилото ја преминало линијата.

Овој метод е особено корисен кога преминувањето се случило помеѓу фрејмови или кога има краткотрајни загуби на детекција, како при шум или непостојани детекции од YOLO. Се активира само ако возилото не е веќе избројано, има барем 3 фрејмови видено и барем 3 позиции во историјата (линија 207).

Со анализа на историјата на позиции, овој метод овозможува да се детектира crossing и во услови каде примарниот или првиот резервен метод би пропуштиле возило, обезбедувајќи точност и конзистентност во броењето на возилата.

4.5. Алгоритам за чистење на стари tracks (Track Cleanup Algorithm)

Алгоритмот **cleanup_old_tracks()** (линија 219-252) отстранува track-ови кои повеќе не се потребни за следење. За секое возило се проверува дали е активно во тековниот фрејм (линија 230).

- За избројани возила, track-овите се отстрануваат ако centroid-от е далеку од линијата (разлика > 300 пиксели) (линија 234-237).
- За возила кои не се избројани, се користи построг праг (400 пиксели) за да се задржат подолго (линија 238-245).

Овој алгоритам ја одржува меморијата ефикасна, спречува дупликати при броење и овозможува сигурно и прецизно следење на возилата низ целиот видео стрим. Track cleanup е клучен за стабилноста на системот, особено во долги видеа со густ сообраќај, каде track ID-тата би можеле да се повторуваат или centroid-ите да се изгубат.

4.6. Алгоритмот за визуелизација (Visualization Algorithm)

Алгоритмот за визуелизација се состои од три дела:

1. **draw_detections()** (линија 254-315) – исцртува bounding boxes и centroids:
 - За секоја детекција се проверуваат class ID и confidence threshold (линија 269-273),
 - Се пресметува centroid (линија 279-281),
 - Се одредува бојата: зелена за избројани, сина за возила кои не се избројани (линија 288-295).
2. **draw_counting_line()** (линија 51-55) – исцртува хоризонтална линија на позиција line_position, која служи како референтна граница за броење.
3. **draw_counters()** (линија 319-388) – прикажува бројачи во горниот лев агол:
 - Се користи draw_text_with_box() за текст со полупровидна позадина,
 - Се прикажуваат вкупен број и броеви по тип на возило, со различни големини и бои за подобра читливост.

Визуелизацијата овозможува директно да се проверува точноста на детекциите, centroid-ите и броењето, како и да се идентификуваат потенцијални грешки или пропуштени возила. Боите и бројачите овозможуваат лесна и интуитивна анализа на тековниот сообраќај и перформансите на алгоритмот во реално време.

4.7. Главниот обработувачки циклус (Main Processing Loop)

Главниот циклус е имплементиран во `process_video()` (линија 390-481). За секој фрејм се извршуваат следните чекори:

1. **Читање на фрејм** од видео (линија 425),
2. **Детекција и следење** со `model.track()` (линија 431-441),
3. **Исцртување на линијата за броење** (линија 443),
4. **Ажурирање на следењето и броењето** (линија 445),
5. **Исцртување на детекциите и centroid-ите** (линија 447),
6. **Прикажување на бројачи** (линија 449),
7. **Запишување на фрејм** ако е наведен излезен фајл (линија 456).

Циклусот се повторува додека има фрејмови, со периодични пораки за напредок (линија 451-453).

Главниот обработувачки циклус ја интегрира целата логика на системот – YOLO детекција, track IDs, centroid-слејтинг, line crossing детекција, резервни методи, cleanup и визуелизација – во еден непрекинат поток. Овој редослед овозможува реално време обработка на видео, точност на броењето и прегледност на резултатите, со минимално губење на податоци или повторување на броеви.

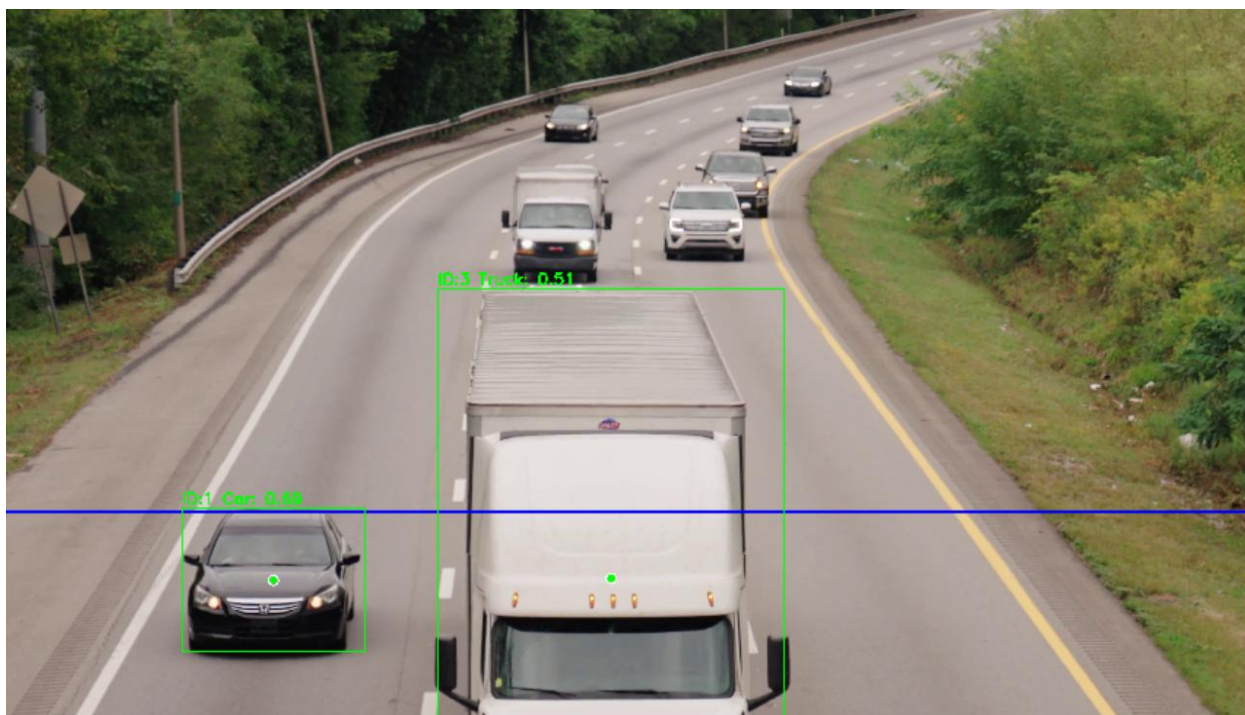
4.8. Алгоритмот за категоризација на возила по тип

Алгоритмот `_increment_vehicle_type_counter()` (линија 41-49) врши категоризација на возилата според **class ID** од YOLO. YOLO користи COCO dataset класификација:

- **class ID 2 – автомобил,**
- **class ID 3 – мотоцикл,**
- **class ID 5 – автобус,**
- **class ID 7 – камион.**

Кога возилото ја преминува линијата, се повикува оваа функција со class ID на возилото и се зголемува соодветниот бројач во `vehicle_count` речникот (линија 33-39).

Овој алгоритам овозможува точна статистика по тип на возило, што е корисно за анализа на сообраќајот. Благодарение на centroid-от и track ID, секое возило се брои точно еднаш, а со категоризацијата се одржуваат одделни бројачи за секој тип возило, како и вкупен бројач, што ја подобрува прецизноста и деталноста на податоците.



Слика бр.2 Категоризација на возило по тип

На сликата (Слика бр.2) се прикажани две детектирања: едно возило од тип **камион (truck)** и едно од тип **автомобил (car)**. Секоја детекција е обележана со bounding box и centroid, што овозможува системот да ги следи движењата на возилата, да утврди дали ја преминале линијата за броење и да ги категоризира точно по тип. Зелената боја го означува возилото што веќе е избројано

4.9.Спречување на дупликатно броење (Double Counting Prevention)

Критичен аспект на алгоритмот е спречувањето на дупликатно броење на истото возило. Ова се постигнува со користење на counted_ids сет (линија 30), кој ги чува сите track IDs на возила што веќе се избројани.

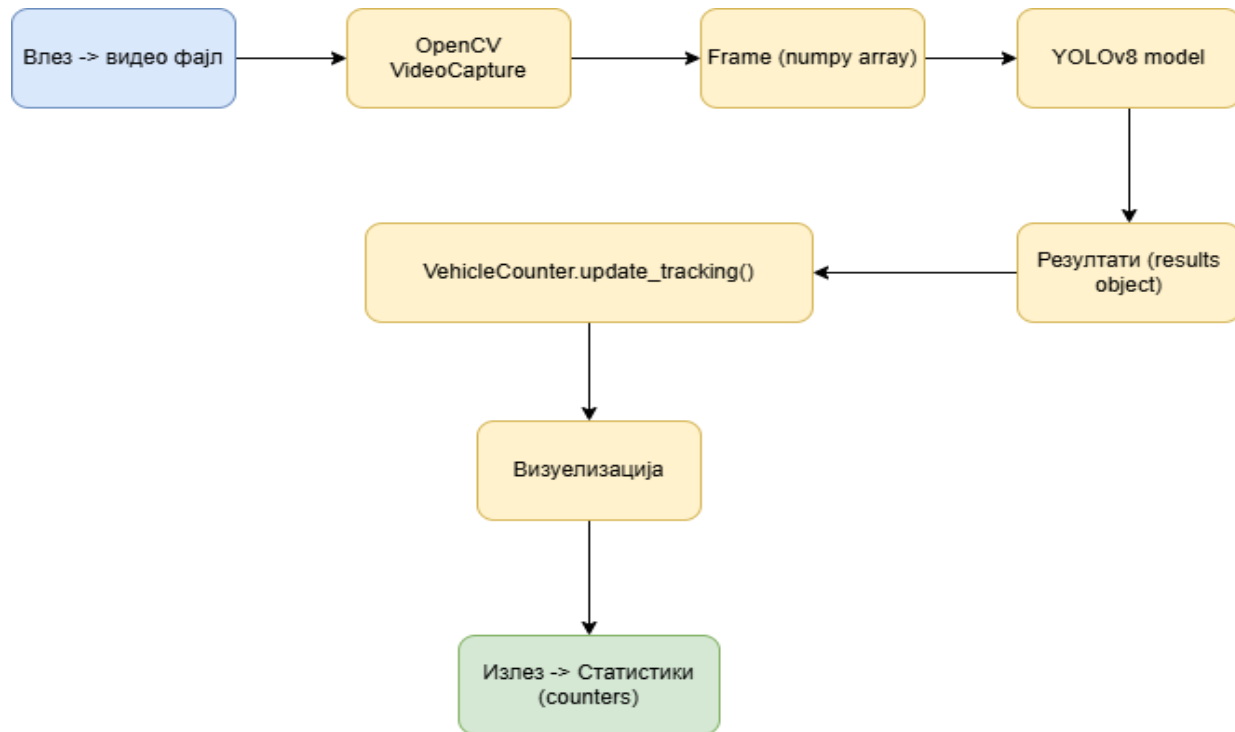
- Кога возилото ја преминува линијата, неговиот track ID се додава во counted_ids (линија 182, 194, 200, 211).
- Пред секоја проверка за преминување се проверува дали ID-то веќе е во сетот (линија 172, 189, 207).

Ова гарантира дека секое возило се брои само еднаш, дури и ако YOLO го детектира повторно или ако резервните методи се активираат повеќепати.

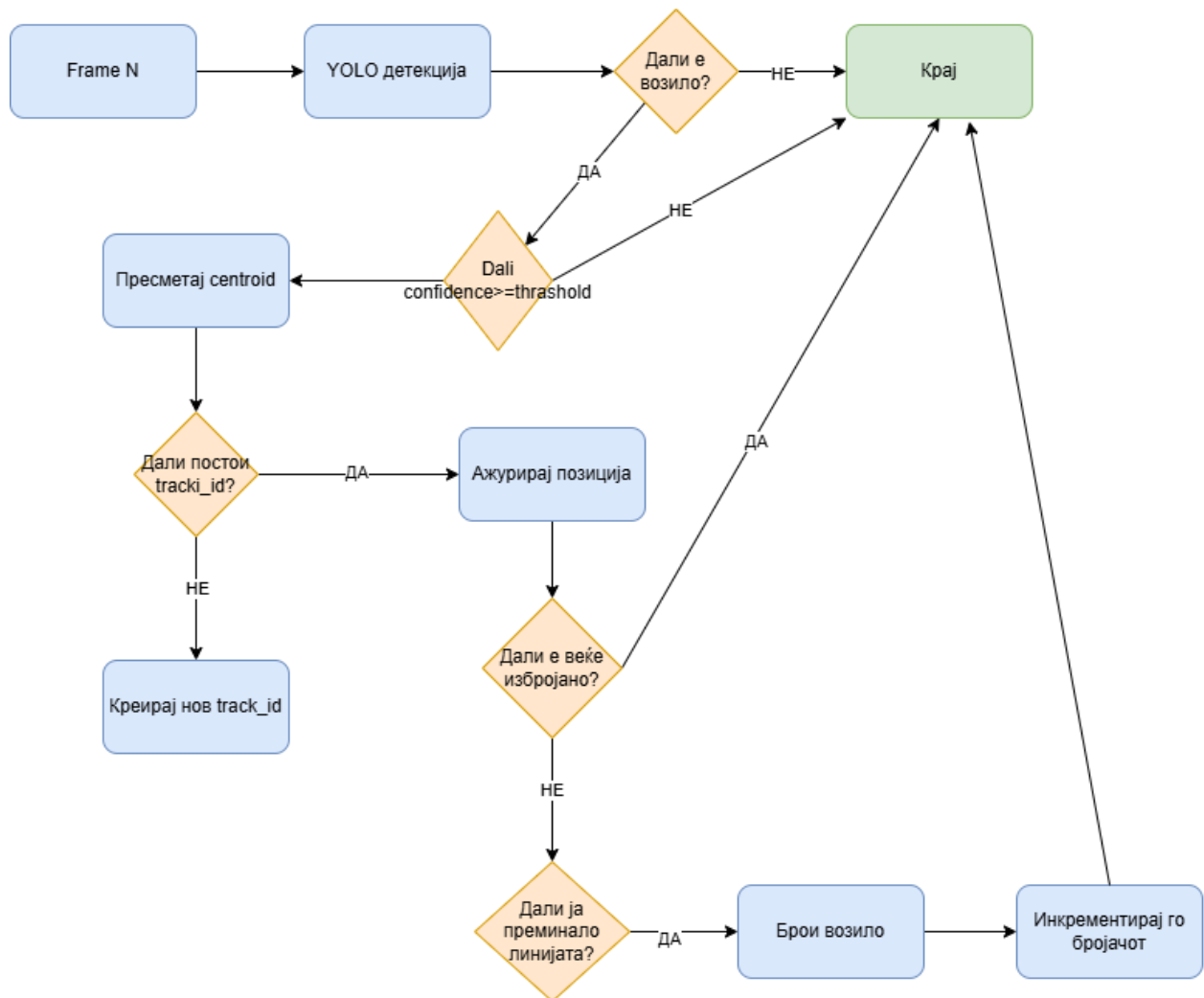
Важно е дека counted_ids се задржува дури и по бришење на track-от од tracked_vehicles (линија 250-252), што спречува повторно броење ако YOLO повторно употреби ист ID подоцна во видеото.

5. Дијаграми

5.1.Data flow diagram



5.2. Activity diagram



6. Референци

<https://yolov8.com/>

<https://arxiv.org/html/2408.15857v1>

<https://www.pexels.com/search/videos/car%20on%20the%20road/>

<https://www.kaggle.com/code/hakim11/vehicles-detection-and-counting>

<https://pixabay.com/videos/search/cars%20highway/>